

C++ 연습 문제: 09. 객체지향 설계 원칙

1. 객체지향 설계(OOD)의 주된 목적으로 가장 올바른 것은? (하)

1. 프로그램의 실행 속도를 물리적인 한계까지 끌어올린다.
2. 코드를 단순히 여러 클래스로 나누어 파일 개수를 늘린다.
3. 변경의 영향 범위를 줄이고, 요구 사항 변경 시 유연하게 확장할 수 있는 구조를 만든다.
4. 모든 데이터와 함수를 하나의 전역 객체에 모아 접근성을 높인다.

2. 다음 중 단일 책임 원칙(SRP)에 대한 설명으로 가장 올바른 것은? (하)

1. 하나의 클래스는 단 하나의 상태(변수)만 가져야 한다.
2. 하나의 클래스는 단 하나의 변경 이유만을 가져야 한다.
3. 하나의 프로그램은 단 하나의 클래스로만 구성되어야 한다.
4. 하나의 함수 안에는 단 하나의 루프문만 존재해야 한다.

3. 플레이어의 기능을 구현할 때, 공격력 계산, 체력 관리, 전투 기록 출력, 효과음 재생을 모두 하나의 클래스에 넣었을 때 가장 크게 위배되는 설계 원칙은 무엇인가? (중)

1. 단일 책임 원칙(SRP)
2. 리스코프 치환 원칙(LSP)
3. 의존성 역전 원칙(DIP)
4. 인터페이스 분리 원칙(ISP)

4. 기존 코드를 수정하지 않고도 새로운 기능을 추가할 수 있도록 설계해야 한다는 원칙은 무엇인가? (하)

1. 단일 책임 원칙(SRP)
2. 개방-폐쇄 원칙(OCP)
3. 인터페이스 분리 원칙(ISP)
4. 의존성 역전 원칙(DIP)

5. 개방-폐쇄 원칙(OCP)을 잘 지킨 설계의 특징으로 올바른 것은? (상)

1. 새로운 기능이 추가될 때마다 기존 클래스의 조건문(if-else, switch)을 모두 찾아 수정해야 한다.
2. 구체적인 클래스에 직접 의존하여 수동으로 객체를 생성한다.
3. 다형성과 인터페이스를 활용하여, 기존 코드를 건드리지 않고 새로운 파생 클래스를 추가하는 방식으로 구조를 확장한다.
4. 모든 멤버 변수를 **public**으로 선언하여 외부에서 자유롭게 값을 대입하도록 만든다.

6. 기반(Base) 클래스의 포인터나 참조를 사용하는 곳에 파생(Derived) 클래스의 객체를 대신 사용해도 프로그램의 논리적 동작이 깨지지 않아야 한다는 원칙은 무엇인가? (하)

1. 리스코프 치환 원칙(LSP)
2. 개방-폐쇄 원칙(OCP)
3. 인터페이스 분리 원칙(ISP)

4. 단일 책임 원칙(SRP)

7. IMovable 인터페이스를 상속받은 StaticDecoration(움직이지 않는 장식물)

클래스가 MoveTo() 함수를 아무 동작도 하지 않거나 예외를 던지도록 구현했다. 이는 어떤 원칙을 위배한 대표적 사례인가? (상)

1. 단일 책임 원칙(SRP)
2. 개방-폐쇄 원칙(OCF)
3. 리스코프 치환 원칙(LSP)
4. 의존성 역전 원칙(DIP)

8. 클라이언트는 자신이 사용하지 않는 메서드에 의존하도록 강제되어서는 안 되며, 크고 방대한 인터페이스보다는 작고 구체적인 여러 개의 인터페이스로 나누어야 한다는 원칙은 무엇인가? (하)

1. 리스코프 치환 원칙(LSP)
2. 개방-폐쇄 원칙(OCF)
3. 단일 책임 원칙(SRP)
4. 인터페이스 분리 원칙(ISP)

9. IGameEntity라는 하나의 큰 인터페이스에 Update(), Render(), ProcessInput(), TakeDamage()가 모두 포함되어 있을 때 발생할 수 있는 문제점으로 가장 올바른 것은? (중)

1. 클래스의 크기가 작아져서 객체 관리가 어려워진다.
2. 입력을 받지 않는 객체나 피해를 입지 않는 객체도 불필요한 함수를 빈 상태로 억지로 구현해야 한다.
3. 프로그램의 실행 속도가 전반적으로 크게 향상된다.
4. 컴파일러가 다이아몬드 상속을 강제로 적용하여 구조를 복잡하게 만든다.

10. 고수준 모듈은 저수준 모듈에 의존해서는 안 되며, 양쪽 모두 추상화(인터페이스)에 의존해야 한다는 설계 원칙은 무엇인가? (하)

1. 인터페이스 분리 원칙(ISP)
2. 단일 책임 원칙(SRP)
3. 의존성 역전 원칙(DIP)
4. 리스코프 치환 원칙(LSP)

11. 전투 서비스 객체가 내부에서 전역 난수 생성기나 콘솔 출력을 직접 사용할 때 발생하는 가장 큰 단점은 무엇인가? (중)

1. 난수 생성 속도가 비정상적으로 느려진다.
2. 시스템 메모리가 심각하게 부족해진다.
3. 외부 환경에 강하게 결합되어 특정 조건에서만 동작하며, 독립적인 단위 테스트가 매우 어려워진다.
4. 컴파일 타임에 타입 검사를 수행할 수 없게 된다.

12. 의존성 역전 원칙(DIP)을 적용하여 기능 결합을 개선하는 가장 올바른 방법은 무엇인가? (상)

1. 코드를 전역 함수로 분리하여 어디서든 바로 호출할 수 있게 만든다.
2. **IRandomSource**와 같은 추상화된 인터페이스를 정의하고, 서비스는 이를 통해서만 기능을 호출하도록 설계한다.
3. 모든 매니저를 싱글톤(**Singleton**) 클래스로 만들어 직접 참조하게 한다.
4. 컴파일 시점에 동작 값을 상수로 고정하여 변동성을 없앤다.

13. 객체가 필요로 하는 외부 의존성을 내부에서 직접 생성하지 않고, 외부에서 인터페이스 형태로 전달받아 사용하는 기법을 무엇이라고 하는가? (중)

1. 캡슐화(**Encapsulation**)
2. 메서드 체이닝(**Method Chaining**)
3. 의존성 주입(**Dependency Injection**)
4. 가상 상속(**Virtual Inheritance**)

14. 생성자 주입(**Constructor Injection**)에 대한 설명으로 올바른 것은? (중)

1. 객체가 생성될 때 필요한 의존성을 매개변수로 전달받아 객체가 처음부터 완전하고 유효한 상태를 가지도록 보장한다.
2. 생성자 내부에서 모든 의존성 객체를 직접 **new**로 동적 할당하여 소유하는 방식이다.
3. 의존성을 전달받지 않고 기본 생성자만으로 임시 객체를 생성한 뒤 추후에 할당한다.
4. 객체 생성 이후에 **public** 멤버 변수를 통해서만 의존성을 주입하는 방식이다.

15. **ILogCombat** 인터페이스를 적용하여 전투 기록을 남기도록 설계했을 때 얻을 수 있는 장점이 아닌 것은? (상)

1. 콘솔에 출력하던 기록을 파일 출력용 클래스로 쉽게 교체할 수 있다.
2. 출력 결과를 코드로 검증하는 **MemoryCombatLog**를 만들어 단위 테스트를 자동화할 수 있다.
3. 전투 로직과 출력 로직이 서로 분리되어 응집도가 높아진다.
4. 모든 로깅 기능이 하나의 전역 변수로 관리되므로 메모리 누수를 원천적으로 방지한다.

16. 객체지향 설계에서 '응집도(**Cohesion**)'와 '결합도(**Coupling**)'에 대한 바람직한 설계 방향은 무엇인가? (하)

1. 응집도는 낮게, 결합도는 높게 유지한다.
2. 응집도는 높게, 결합도는 낮게 유지한다.
3. 둘 다 높게 유지한다.
4. 둘 다 낮게 유지한다.

17. 다음 중 위에서 다룬 객체지향의 5대 핵심 설계 원칙의 앞 글자를 따서 통칭하는 용어는 무엇인가? (하)

1. **SOLID** 원칙
2. **DRY** 원칙
3. **KISS** 원칙
4. **RAII** 원칙

18. 다음 빈칸 (A)에 들어갈 알맞은 낱말은? (중) [(A) 패턴은 프로그램 내에 해당 객체가 단 하나만 존재하도록 보장하며 전역적으로 접근할 수 있게 해준다. 그러나 여러 모듈이 이 전역 상태에 강하게 결합되어 테스트와 확장을 어렵게 하므로 무분별한 사용을 지양해야 한다.]

1. 팩토리(Factory)
2. 빌더(Builder)
3. 싱글톤(Singleton)
4. 어댑터(Adapter)

19. 프로그램 테스트를 위해 실제 동작하는 객체 대신, 원하는 고정된 값을 반환하거나 상태를 흉내 내도록 껍데기만 구현한 객체(예: 항상 같은 난수를 반환하는 `FixedRandomSource`)를 부르는 일반적인 용어는 무엇인가? (상)

1. 불변 객체(Immutable Object)
2. 모의 객체(Mock Object / Test Double)
3. 래퍼 객체(Wrapper Object)
4. 동적 객체(Dynamic Object)

20. 다음 중 인터페이스 분리 원칙(ISP)을 올바르게 적용한 사례로 가장 적절한 것은? (상)

1. 하나의 인터페이스에 게임 내 모든 동작(이동, 공격, 렌더링, 저장 등)을 모아 여러 객체가 상속받게 한다.
2. 기능이 없어도 파생 클래스에서 모든 인터페이스 함수를 억지로 재정의하여 내부에서 예외를 던지도록 한다.
3. `IUpdatable`, `IRenderable`, `IDamageable` 등 역할별로 잘게 쪼개어 객체가 자신에게 진짜 필요한 기능만 상속받아 구현하게 한다.
4. 인터페이스를 전혀 사용하지 않고 오직 구체 클래스들의 다중 상속만으로 코드를 결합한다.